Dev Workflow com Claude Code

Setup, guardrails e playbook por tarefa

Uma adaptação para o Claude Code de um workflow de desenvolvimento com IA pensado originalmente para o OpenCode: mantém o que agrega, troca pelo recurso nativo o que já existe e descarta o que só custa contexto.

Versão 1.0 · outubro de 2026
Acompanha o template em scripts/new-project.sh

Sumário

1 A ideia em uma página
2 O que o Claude Code já resolve
3 Setup da máquina (uma vez)
4 MCPs: o que vai onde
5 Task Master: do PRD ao backlog
6 Memória: três camadas, uma regra
7 O fluxo de cada tarefa
8 Guardrails
9 Gestão de contexto
10 Usando o template
11 Checklist
12 Problemas comuns
13 O que ficou de fora (e quando reconsiderar)

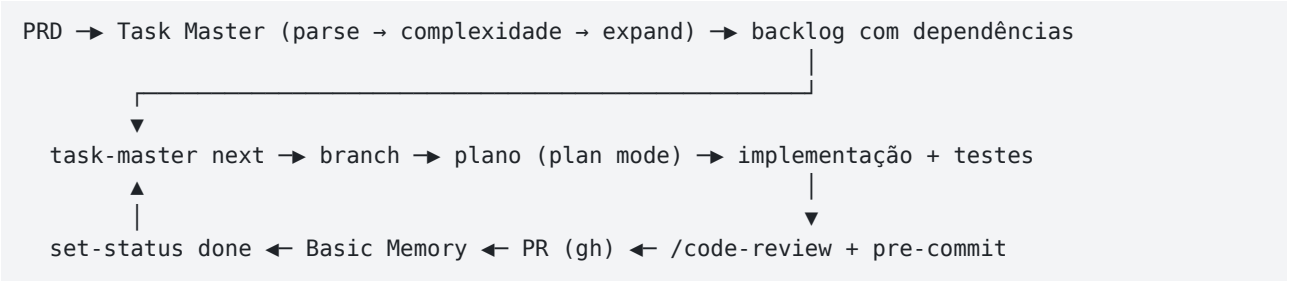
1. A ideia em uma página

Um agente de código sem processo produz código rápido e inconsistente: cada sessão começa do zero, decisões se perdem, testes ficam para depois e commits caem direto na `main`. Este guia transforma o workflow em **instrução explícita e verificável**: o que o agente deve fazer fica escrito no repositório, e o que ele não pode fazer é bloqueado por ferramenta, não por boa vontade.

Quatro princípios

- **O processo mora no repositório.** `CLAUDE.md` e `dev-workflow.md` são lidos em toda sessão; quem clonar o projeto herda o mesmo comportamento.
- **Contexto é o recurso mais caro.** Tudo que entra no prompt em toda sessão precisa se pagar. Ferramenta instalada "por via das dúvidas" é custo sem retorno.
- **Guardrail automático vence disciplina.** Bloquear commit na `main` com hook é melhor que lembrar o agente de não fazer isso.
- **Decisão importante vira arquivo versionado.** Se não está no Git, não existe para o próximo dev (ou para a próxima sessão).

O ciclo



> **Para quem é este guia**
> Para quem vai usar o Claude Code como agente principal e quer um padrão reaproveitável entre projetos. Ele acompanha um template: `scripts/new-project.sh` cria um projeto novo já configurado.

2. O que o Claude Code já resolve

O workflow de referência foi montado para o OpenCode, que depende de plugins e MCPs para várias coisas que o Claude Code traz de fábrica. A tabela mostra a decisão para cada peça.

Necessidade	No guia OpenCode	No Claude Code	Decisão
Ver onde o contexto está sendo gasto	Plugin TokenScope	`/context` e `/cost`	Nativo
Instruções carregadas em toda sessão	`AGENTS.md`	`CLAUDE.md` (com imports via `@arquivo`)	Nativo
Pedir um plano antes de codar	Pedir no prompt	Plan mode (Shift+Tab)	Nativo

Necessidade	No guia OpenCode	No Claude Code	Decisão
Revisar o diff com IA	Pedir no prompt	`/code-review`	Nativo
Continuar trabalho entre sessões	Handover manual	Compactação automática, `/compact`, `/resume`	Nativo (handover só em troca de pessoa)
Contexto pessoal entre projetos	Engram	Memória automática do Claude Code	Nativo
Comprimir saída de comandos	RTK	Saídas longas já são truncadas	Descartado
Navegação semântica (LSP)	Serena	Busca e leitura nativas	Descartado por ora
Proibir ações perigosas	—	`permissions.deny` em `.claude/settings.json`	Nativo
PRD → tarefas com dependências	Task Master	Não existe	Mantido
Docs atuais de bibliotecas	Context7	Busca web (genérica, mais cara)	Mantido
Decisões versionadas do projeto	Basic Memory	Não existe por projeto/Git	Mantido
Código real no GitHub	Octocode	`gh search code` cobre parte	Opcional
Bloquear commit na main	pre-commit	—	Mantido

3. Setup da máquina (uma vez)

Testado em Ubuntu. No macOS troque `apt` por `brew`; no Windows use WSL2 e mantenha os projetos em `/home/...`, nunca em `/mnt/c`.

Base

```
sudo apt install -y git gh
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
curl -LsSf https://astral.sh/uv/install.sh | sh        # uv + uvx (Python)
gh auth login                                          # GitHub.com → HTTPS → navegador
```

Ferramentas do workflow

```
npm install -g task-master-ai        # orquestrador de tarefas
uv tool install basic-memory         # memória versionada por projeto
uv tool install pre-commit           # hooks de commit
```

MCPs globais (escopo de usuário)

```
claude mcp add --scope user context7 -- npx -y @upstash/context7-mcp
claude mcp add --scope user octocode -- npx -y octocode-mcp@latest     # opcional
```

Confira com `claude mcp list` (ou `/mcp` dentro de uma sessão). Se `basic-memory` ou `pre-commit` não forem encontrados, adicione `~/.local/bin` ao `PATH`.

4. MCPs: o que vai onde

A regra é simples: **ferramenta que não depende do projeto fica no escopo de usuário; ferramenta que guarda estado do projeto fica no** `.mcp.json` do repositório.

MCP	Escopo	Onde está configurado	Por quê
Context7	Usuário	`~/.claude.json`	Documentação de bibliotecas vale para qualquer projeto
Octocode	Usuário	`~/.claude.json`	Busca no GitHub não tem estado de projeto
Task Master	Projeto	`.mcp.json`	Lê e escreve `.taskmaster/` do repositório
Basic Memory	Projeto	`.mcp.json` com `--project <nome>`	Cada projeto enxerga só as próprias memórias

Formato no Claude Code

O `.mcp.json` do Claude Code não usa o formato do OpenCode. A tradução é mecânica:

OpenCode (`opencode.json`)	Claude Code (`.mcp.json`)
`"type": "local"`	`"type": "stdio"`
`"command": ["uvx", "basic-memory", "mcp"]`	`"command": "uvx"` + `"args": ["basic-memory", "mcp"]`
`"environment": {...}`	`"env": {...}`
`"enabled": true`	Não existe: aprovação por `enabledMcpjsonServers` ou na primeira abertura

```
{
  "mcpServers": {
    "task-master-ai": {
      "type": "stdio", "command": "npx", "args": ["-y", "task-master-ai"],
      "env": { "TASK_MASTER_TOOLS": "core" }
    },
    "basic-memory": {
      "type": "stdio", "command": "uvx",
      "args": ["basic-memory", "mcp", "--project", "meu_projeto"]
    }
  }
}
```

> **Nada de chave no .mcp.json**
> O `.mcp.json` vai para o Git. O Task Master lê as chaves do `.env` do projeto, que fica no `.gitignore`. O template já vem assim.

5. Task Master: do PRD ao backlog

É a única peça sem equivalente no Claude Code: transforma um PRD em tarefas pequenas, com dependências e complexidade estimada, que **sobrevivem entre sessões**. Com isso, `task-master next` sempre sabe o que vem a seguir.

Modelos sem chave de API

O Task Master tem um provider `claude-code` que roda o próprio Claude Code e usa a sua assinatura. Com ele, `main` e `fallback` não precisam de chave. Só o `research` (Perplexity) precisa, e é opcional.

```
task-master models --set-main opus --claude-code
task-master models --set-fallback sonnet --claude-code
task-master models --set-research sonar-pro          # requer PERPLEXITY_API_KEY no .env
task-master models                                   # confere a configuração
```

Fluxo inicial do projeto

1. Escreva o PRD em `.taskmaster/docs/prd.txt`. Antes do parse, peça ao Claude para apontar ambiguidades.
2. `task-master parse-prd .taskmaster/docs/prd.txt`: gera as tarefas de primeiro nível.
3. `task-master analyze-complexity --research`: dá nota de 1 a 10 e recomenda quantas subtarefas cada uma merece.
4. `task-master complexity-report`: revise antes de expandir.
5. `task-master expand --all`: quebra as tarefas usando as recomendações do relatório.

A ordem importa: o `expand` lê o relatório de complexidade. Expandir antes de analisar gera subtarefas em número fixo, sem relação com a dificuldade real.

Economia de contexto

- `TASK_MASTER_TOOLS=core` expõe só 7 ferramentas MCP. Suba para `standard` apenas se sentir falta.
- O `task-master init` cria um `.taskmaster/CLAUDE.md` com cerca de 1.800 palavras e sugere importá-lo no `CLAUDE.md`. **Não importe.** O template traz um resumo de 5 linhas e aponta para o arquivo completo, para ser lido só quando necessário.

6. Memória: três camadas, uma regra

O Claude Code tem memória própria, e o Basic Memory é outra. Sem uma regra clara, o agente salva no lugar errado, e foi exatamente isso que aconteceu no primeiro teste deste setup.

Camada	Onde fica	Quem lê	O que guardar
`CLAUDE.md` + `dev-workflow.md`	Repositório (Git)	Toda sessão, sempre	Regras e processo: curto e estável
Basic Memory	`memories/` no repositório (Git)	Sob demanda, por busca	Decisões de arquitetura, convenções, lições aprendidas
Memória automática	`~/.claude/projects/...` (local)	Só você	Preferências pessoais de trabalho

> **A regra**
> Decisão que afeta o projeto vai no Basic Memory. Ela fica escrita no `CLAUDE.md` do template, então o agente não precisa adivinhar.

Como pedir

```
"Salva no Basic Memory uma decisão de arquitetura: usamos Hono em vez de Express.
 Contexto: API pequena, deploy em edge. Trade-off: ecossistema menor de middlewares."
```

Decisão + contexto + alternativas descartadas. Sem o porquê, a nota não ajuda ninguém daqui a seis meses. Para ver as notas como grafo, abra a pasta `memories/` como vault no Obsidian. Se editar arquivos à mão, rode `basic-memory sync`.

7. O fluxo de cada tarefa

Este é o conteúdo do `dev-workflow.md`, importado pelo `CLAUDE.md`. Para começar, basta dizer "pega a próxima tarefa".

Etapa	O que acontece	Ferramenta
1. Entender	Ler a tarefa e os critérios de aceite; se estiverem vagos, perguntar. Buscar decisões anteriores e documentação atual.	`task-master show`, Basic Memory, Context7, Octocode
2. Preparar	Árvore limpa, uma branch por tarefa, status em andamento, testes rodando antes de mexer.	`git checkout -b feat/<id>-...`, `set-status`
3. Planejar	Plano com arquivos, abordagem e testes. Nada de código antes da aprovação.	Plan mode (Shift+Tab)
4. Implementar	Passos pequenos, teste junto do código, progresso registrado na subtarefa.	`update-subtask`
5. Validar	Testes, lint e tipos verdes. Revisão do diff por IA.	`/code-review`
6. Entregar	Conventional Commits citando a tarefa; PR pelo `gh`.	`gh pr create`

Etapa	O que acontece	Ferramenta
7. Registrar	Decisões no Basic Memory, README atualizado, tarefa concluída.	`set-status done`

Dicas que fazem diferença

- Use `/clear` entre tarefas. O contexto da tarefa anterior só atrapalha a próxima.
- Para tarefas independentes em paralelo, use git worktrees: uma sessão do Claude Code por worktree.
- Peça buscas amplas no código a um subagente: o resultado volta resumido e o contexto principal fica limpo.

8. Guardrails

Duas camadas complementares: o **pre-commit** protege o repositório contra qualquer ferramenta ou pessoa; as **permissões do Claude Code** impedem o agente de contornar a primeira.

pre-commit (`.pre-commit-config.yaml`)

```
- repo: local
  hooks:
    - id: block-main
      name: Block direct commits to main/master
      entry: bash -c 'branch=$(git symbolic-ref --short HEAD);
             if [ "$branch" = main ] || [ "$branch" = master ]; then exit 1; fi'
      language: system
      pass_filenames: false
      always_run: true
```

O template também traz checagens de espaço em branco, YAML e arquivos grandes, além de um hook `run-tests` comentado. **Descomente esse hook assim que o projeto tiver testes.**

Permissões (`.claude/settings.json`)

```
"deny": [
  "Bash(git commit --no-verify:*)", "Bash(git commit -n:*)",
  "Bash(git push --force:*)", "Bash(git push -f:*)", "Bash(git push origin main:*)",
  "Read(./.env)", "Read(./.env.local)"
]
```

> **Por que bloquear --no-verify**
> Hook que o agente pode pular é sugestão, não guardrail. Sem esse deny, um commit barrado pelo pre-commit pode virar um `--no-verify` na tentativa seguinte.

O único commit sem hook é o inicial, feito pelo `scripts/setup.sh`. Depois dele, a `main` só recebe mudanças via PR.

9. Gestão de contexto

Situação	O que fazer
Quer saber onde o contexto está indo	`/context` (por categoria) e `/cost` (tokens e custo da sessão)
Terminou uma tarefa	`/clear` antes da próxima
Sessão longa, mesma tarefa	`/compact`, ou deixe a compactação automática agir
Retomar depois	`/resume` para voltar à sessão anterior
Passar o trabalho para outra pessoa	Handover: feito, arquivos, decisões, testes, pendências, próxima tarefa
Busca ampla no código	Delegar a um subagente
`CLAUDE.md` crescendo	Mover detalhes para arquivos lidos sob demanda; deixar no `CLAUDE.md` só regras

Referência de tamanho: o `CLAUDE.md` do template, com o `dev-workflow.md` importado, tem cerca de 400 palavras. O arranjo padrão do `task-master init` passava de 1.800, só com a referência do Task Master.

10. Usando o template

```
cd ~/Documents/dev_workflow_course
scripts/new-project.sh ~/projetos/meu-app            # nome = nome da pasta
scripts/new-project.sh ~/projetos/meu-app meu_app    # ou nome explícito
```

O script copia o template sem histórico, segredos, memórias nem tarefas. Depois registra o projeto no Basic Memory, ajusta o `--project` no `.mcp.json`, cria o `.env`, roda `git init -b main`, instala o pre-commit, faz o commit inicial e avisa se o Context7 não estiver configurado. Se o projeto foi criado por outro caminho (por exemplo, `gh repo create --template`), rode só `scripts/setup.sh`.

Estrutura

```
meu-app/
├── CLAUDE.md                 regras + import do dev-workflow.md
├── dev-workflow.md           playbook por tarefa (preencha convenções e comandos)
├── README.md
├── .mcp.json                 Task Master + Basic Memory
├── .claude/settings.json     permissões e MCPs aprovados
├── .pre-commit-config.yaml   bloqueio da main + checagens
├── .env.example → .env       PERPLEXITY_API_KEY (opcional)
├── memories/                 Basic Memory (versionado)
├── .taskmaster/              config, docs/prd.txt, templates
└── scripts/                  new-project.sh, setup.sh
```

Depois de criar

1. Preencher as convenções e os comandos (testes, lint, build) no `dev-workflow.md`.
2. Ativar o hook `run-tests` no `.pre-commit-config.yaml` quando houver testes.
3. Escrever o PRD e rodar o fluxo da seção 5.
4. Abrir o Claude Code, conferir `/mcp` e pedir a primeira tarefa.

11. Checklist

	Item	Como verificar
☐	Node 18+, uv, git, gh	`node -v`, `uv --version`, `gh auth status`
☐	task-master, basic-memory, pre-commit	`--version` de cada um
☐	Context7 global	`claude mcp list` mostra context7 ✓
☐	Projeto criado pelo template	`git log` mostra o commit inicial
☐	Basic Memory conectado	`/mcp` mostra basic-memory ✓
☐	Task Master com claude-code	`task-master models`
☐	Hook bloqueia a main	commit na main falha com mensagem
☐	Convenções preenchidas	`dev-workflow.md` sem `<preencher>`
☐	Hook de testes ativo	`pre-commit run --all-files` roda os testes
☐	Primeira memória salva	arquivo `.md` aparece em `memories/`

12. Problemas comuns

Sintoma	Causa provável e solução
MCP do projeto não aparece em `/mcp`	O Claude Code só lê o `.mcp.json` ao iniciar: reinicie. Confira se o servidor está em `enabledMcpjsonServers` e se o JSON é válido (`python3 -m json.tool .mcp.json`).
Editor não mostra mudança no `.mcp.json`	O buffer está desatualizado: reabra o arquivo.
Basic Memory: "project not found"	O nome em `--project` precisa ser igual ao de `basic-memory project list`, e o caminho registrado precisa ser absoluto.
Memória salva no lugar errado	O agente usou a memória automática. Verifique a regra no `CLAUDE.md` e peça explicitamente "no Basic Memory".
`--research` falha	Falta `PERPLEXITY_API_KEY` no `.env`, ou rode sem `--research`.
Fallback do Task Master pede chave	Ele ainda aponta para a API da Anthropic: `task-master models --set-fallback sonnet --claude-code`.
pre-commit: "mapping values are not allowed"	Há `:` (dois-pontos + espaço) dentro de um `entry` sem aspas no YAML. Reescreva a mensagem.
`basic-memory`/`pre-commit` não encontrado	Adicione `~/.local/bin` ao `PATH`.

13. O que ficou de fora (e quando reconsiderar)

Ferramenta	Por que não entrou	Reconsidere quando
Serena	Num projeto pequeno, a busca nativa resolve. A configuração de referência baixa o código do GitHub, sem versão fixa, a cada inicialização.	O código passar de algumas dezenas de milhares de linhas ou houver muitos refactors entre arquivos. Fixe uma versão.
RTK	Instala um hook global que reescreve comandos, e o ganho sobre o truncamento nativo é pequeno.	`/context` mostrar saídas de comando dominando o contexto.
Engram	A memória automática do Claude Code cobre o contexto pessoal.	Você usar vários agentes diferentes e quiser uma memória comum entre eles.
TokenScope	`/context` e `/cost` são nativos.	Nunca, no Claude Code.
Perplexity	Opcional: só serve para o `--research` do Task Master.	PRDs em domínios que mudam rápido (libs novas, APIs instáveis).

Fechamento
O workflow não vale pelas ferramentas, e sim pelo ciclo: tarefa clara, branch isolada, plano aprovado, validação automática e decisão registrada. Ferramenta que não sustenta esse ciclo é custo de contexto.